Overview of Disk Scheduling Algorithms

FCFS, SSTF, SCAN, C-SCAN and LOOK

Prashant Jadav

DEPSTAR - CHARUSAT

Why Disk Scheduling Is Needed

- Multiple processes may generate disk I/O requests at the same time.
- The disk cannot necessarily serve all requests simultaneously.
- The operating system maintains a **request queue**.
- A disk scheduling algorithm decides the order in which requests are serviced.

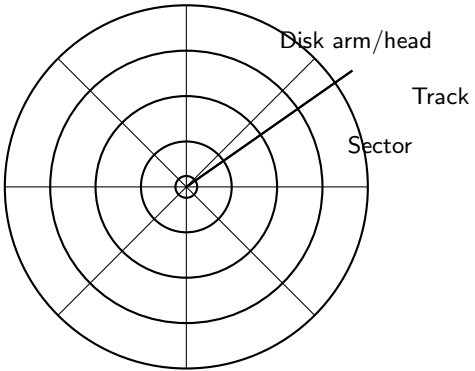
Main objectives

- Reduce disk-head movement
- Reduce average response/waiting time
- Increase I/O throughput
- Provide reasonable fairness
- Avoid starvation where possible

Key idea

For a traditional HDD, reducing unnecessary head movement can substantially reduce access time.

Magnetic Disk Organization



- A **track** is a circular path on the disk surface.
- A **sector** is a subdivision used to store data.
- The disk head moves between tracks to access requested data.

Disk Access Time

For a magnetic disk:

$$T_{\text{access}} = T_{\text{seek}} + T_{\text{rotational}} + T_{\text{transfer}}$$

Seek time: Time to move the disk head to the required track.

Rotational latency: Time waiting for the required sector to rotate under the head.

Transfer time: Time to transfer the requested data.

Disk scheduling focus

Most classical disk scheduling algorithms primarily attempt to reduce **head movement / seek distance**.

Scope

These algorithms are especially relevant to traditional HDDs. SSDs have no mechanical disk head, so seek-distance optimization has much less significance.

Terminology Used in Disk Scheduling

- **Current head position:** Track/cylinder where the disk head is currently located.
- **Request queue:** Pending disk requests waiting for service.
- **Seek distance:** Absolute difference between two cylinder positions.

$$\text{Head movement} = |C_{\text{new}} - C_{\text{current}}|$$

Example

If the head moves from cylinder 50 to cylinder 90:

$$\text{Movement} = |90 - 50| = 40 \text{ cylinders}$$

Common Numerical Example

We will use the same example to understand all five algorithms.

Given

- Disk cylinders: 0 to 199
- Initial head position: **53**
- Pending requests:

$$98,\quad 183,\quad 37,\quad 122,\quad 14,\quad 124,\quad 65,\quad 67$$

- For SCAN and C-SCAN, assume the initial direction is **toward higher-numbered cylinders**.
- We will calculate the service order and total head movement.

FCFS: First-Come, First-Served

Idea:

- Serve requests exactly in their arrival order.
- No attempt is made to optimize head movement.
- Very simple to implement.

For the example:

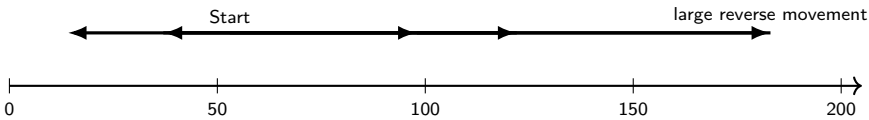
$$53 \to 98 \to 183 \to 37 \to 122 \to 14 \to 124 \to 65 \to 67$$

$$\begin{aligned} M &= |98-53| + |183-98| + |37-183| + |122-37| \\ &\quad + |14-122| + |124-14| + |65-124| + |67-65| \\ &= 640 \text{ cylinders} \end{aligned}$$

Observation

FCFS is simple and preserves arrival order, but the total head movement can be very large.

FCFS: Visualizing Head Movement



Problem

The head repeatedly moves back and forth across the disk instead of following a locality-aware path.

SSTF: Shortest Seek Time First

Idea:

- Select the pending request closest to the current head position.
- Similar in spirit to choosing the nearest available request.
- Usually reduces immediate seek distance.

Starting at 53:

$$53 \to 65 \to 67 \to 37 \to 14 \to 98 \to 122 \to 124 \to 183$$

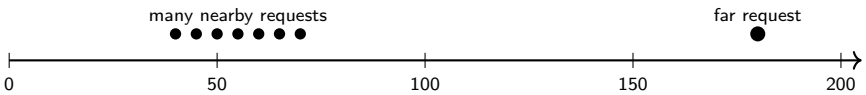
$$\begin{aligned} M &= 12 + 2 + 30 + 23 + 84 + 24 + 2 + 59 \\ &= 236 \text{ cylinders} \end{aligned}$$

Observation

SSTF greatly reduces movement for this particular request set compared with FCFS.

SSTF: Important Limitation

- SSTF makes a locally shortest choice.
- A request far away may repeatedly be bypassed by requests arriving near the current head position.
- This can lead to **starvation** or very long waiting time.



Key trade-off

SSTF can provide low average seek distance, but it does not inherently guarantee fairness.

SCAN: The Elevator Algorithm

Idea:

- Move the disk head in one direction.
- Service requests encountered along the way.
- Continue toward the end of the disk.
- Reverse direction and service requests on the return path.

For the example, direction is initially upward:

$$53 \rightarrow 65 \rightarrow 67 \rightarrow 98 \rightarrow 122 \rightarrow 124 \rightarrow 183 \rightarrow 199$$

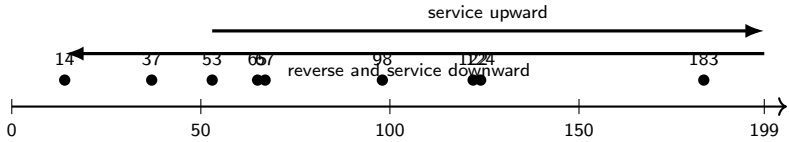
$$\rightarrow 37 \rightarrow 14$$

$$M = (199 - 53) + (199 - 14) = 331$$

Observation

SCAN provides a more systematic service order and reduces the possibility of indefinite postponement.

SCAN: Visual Model



Analogy

SCAN behaves like an elevator: it serves requests while moving in one direction, then reverses.

C-SCAN: Circular SCAN

Idea:

- Service requests while moving in one direction only.
- After reaching the end, return to the beginning.
- Do not service requests while making the return.
- Continue servicing in the original direction.

For the example:

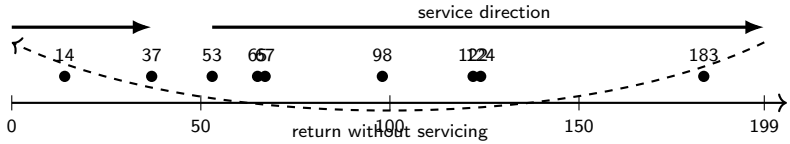
$$53 \to 65 \to 67 \to 98 \to 122 \to 124 \to 183 \to 199$$
$$\to 0 \to 14 \to 37$$

$$M = (199 - 53) + 199 + (37 - 0) = 382$$

Observation

C-SCAN provides a more uniform waiting-time pattern across the disk.

C-SCAN: Visual Model



Difference from SCAN

SCAN services requests in both directions; C-SCAN services requests in one direction and uses a circular return.

LOOK: An Improved SCAN

Idea:

- LOOK follows the SCAN concept.
- It reverses direction at the **last pending request**.
- It does not travel all the way to the physical end of the disk when no request exists there.

For the example:

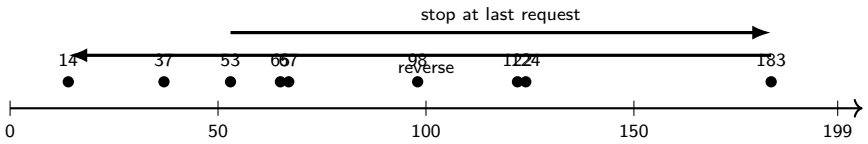
$$53 \rightarrow 65 \rightarrow 67 \rightarrow 98 \rightarrow 122 \rightarrow 124 \rightarrow 183$$
$$\rightarrow 37 \rightarrow 14$$

$$M = (183 - 53) + (183 - 14) = 299$$

Why "LOOK"?

The algorithm "looks" ahead to determine whether there are pending requests before moving farther.

LOOK: Visual Model



SCAN vs. LOOK

SCAN goes to the disk boundary before reversing; LOOK reverses when there are no more requests in the current direction.

C-LOOK Disk Scheduling Algorithm

Step 1: Arrange requests in sorted order

$$14,\ 37,\ 65,\ 67,\ 98,\ 122,\ 124,\ 183$$

Step 2: Determine the service sequence

C-LOOK services requests in the current direction, jumps from the highest pending request to the lowest pending request, and continues in the same direction.

$$\boxed{53 \rightarrow 65 \rightarrow 67 \rightarrow 98 \rightarrow 122 \rightarrow 124 \rightarrow 183 \rightarrow 14 \rightarrow 37}$$

Step 3: Calculate total head movement

Head Movement	**Distance**
$53 \rightarrow 65$	12
$65 \rightarrow 67$	2
$67 \rightarrow 98$	31
$98 \rightarrow 122$	24
$122 \rightarrow 124$	2
$124 \rightarrow 183$	59
$183 \rightarrow 14$	169
$14 \rightarrow 37$	23
Total	**322 cylinders**

FCFS vs. SSTF vs. SCAN vs. C-SCAN vs. LOOK vs. C-LOOK

Algorithm	Selection Principle	Strength	Main Concern
FCFS	Arrival order	Simple and fair by order	High head movement
SSTF	Closest pending request	Often low seek distance	Starvation possible
SCAN	Sweep to disk boundary, then reverse	Good balance of fairness and movement	Travels toward boundary
C-SCAN	Sweep to boundary, then wrap around	More uniform waiting time	Circular return movement
LOOK	Reverse at last pending request	Avoids unnecessary boundary travel	Depends on direction and workload
C-LOOK	Jump between extreme pending requests	Avoids boundary travel; one-way service	Circular jump can add movement

Important

No algorithm is universally optimal. The choice depends on the request workload and performance objectives.

Worked Example: Total Head Movement

Disk cylinders: 0–199
Initial head: 53, initially moving toward higher cylinders
Requests:

98, 183, 37, 122, 14, 124, 65, 67

Algorithm	Total Movement	Main Service Behavior
FCFS	640	Arrival order
SSTF	236	Nearest request first
SCAN	331	Sweep to boundary, then reverse
C-SCAN	382	Sweep to boundary, then wrap
LOOK	299	Reverse at last pending request
C-LOOK	322	Jump between extreme requests

Interpret Carefully

These results depend on the request sequence, initial head position, direction, and boundary-handling convention. They do not establish a universally superior algorithm.

How to Solve Disk Scheduling Problems

1. Write the initial head position.
2. Write the complete pending request queue.
3. Identify the rule of the selected algorithm.
4. Determine the service sequence.
5. Calculate each individual head movement.
6. Add all movements:

$$M = \sum_{i=1}^{n} |C_i - C_{i-1}|$$

7. For SCAN/C-SCAN/LOOK, clearly state the initial direction.

Exam tip

Always draw a small cylinder-number line. It makes the service sequence and movement calculation much easier to verify.

Fairness and Starvation

- **Fairness** means requests should receive reasonable service opportunities.
- **Starvation** occurs when a request waits indefinitely or for an excessively long time.

Algorithm	Fairness Tendency	Reason
FCFS	High	Requests retain their arrival order.
SSTF	Lower	Nearby requests may repeatedly be selected, delaying distant requests.
SCAN	Good	Requests are served during sweeps in both directions.
C-SCAN	Good/predictable	Requests are served in one direction, providing more uniform waiting-time behavior.
LOOK	Good	Requests are served during directional sweeps without unnecessary boundary travel.
C-LOOK	Good	Requests are served in one direction, with a jump between the highest and lowest pending requests.

Design Principle

Minimizing average seek distance is not enough; an OS must also consider fairness, starvation risk, and response-time distribution.

Choosing an Algorithm: What Matters?

- **Workload:** Random, sequential, bursty, or highly localized?
- **Fairness:** Can long waits be tolerated?
- **Throughput:** How many requests must be completed per unit time?
- **Latency:** Is individual response time critical?
- **Device technology:** HDD or SSD?
- **Queue behavior:** Are requests arriving continuously?

System-design perspective

A policy that works well for a heavily loaded HDD may not provide the same benefit on an SSD because the underlying hardware costs are different.

HDD Scheduling vs. SSD Scheduling

Traditional HDD

- Mechanical head movement
- Seek time matters
- Rotational latency matters
- Classical algorithms are highly relevant

SSD

- No mechanical disk head
- No seek movement
- High internal parallelism
- Scheduling focuses more on queueing, fairness, parallelism and device behavior

Key point

FCFS/SSTF/SCAN/C-SCAN/LOOK are primarily classical **HDD disk scheduling** concepts.

Real-World Analogy

- **FCFS:** A queue at a service counter – serve people in arrival order.
- **SSTF:** A delivery person serves the nearest pending destination first.
- **SCAN:** An elevator serves requests while moving upward, then reverses.
- **C-SCAN:** A circular shuttle serves passengers in one direction, then returns to the start.
- **LOOK:** An elevator reverses when there are no more requested floors in the current direction instead of going to the building's top/bottom.

Why analogies help

All five algorithms answer the same fundamental question:

"Which pending request should be served next?"

Quick Check

1. Why can FCFS produce large disk-head movement?
2. How does SSTF select the next request?
3. Why can SSTF cause starvation?
4. Explain the elevator analogy for SCAN.
5. What is the main difference between SCAN and C-SCAN?
6. What does LOOK avoid that SCAN may perform?
7. Why must the initial direction be specified for SCAN and C-SCAN?
8. Why are these algorithms less relevant to SSD seek optimization?

Higher-Order Problem

Problem

A disk has cylinders 0–199. The initial head position is 53 and the request queue is:

$$98,\ 183,\ 37,\ 122,\ 14,\ 124,\ 65,\ 67$$

Assume the initial direction is toward higher-numbered cylinders.

1. Determine the service sequence using SCAN.
2. Calculate the total head movement.
3. Repeat using LOOK.
4. Explain why the total movement differs.
5. Discuss which algorithm would be more appropriate if fairness is more important than minimizing immediate seek distance.

Do not choose an algorithm only from its average movement; analyze the workload and design objective.

Key Takeaways

- **FCFS:** simplest; follows arrival order.
- **SSTF:** selects the nearest request; may cause starvation.
- **SCAN:** moves like an elevator and reverses direction.
- **C-SCAN:** services in one direction and returns circularly.
- **LOOK:** behaves like SCAN but reverses at the last pending request.
- Total head movement is calculated using absolute cylinder differences.
- For SCAN/C-SCAN/LOOK, the initial direction must be specified.
- Classical disk scheduling is mainly associated with mechanical HDDs.

Schedule the requests; minimize unnecessary movement; preserve fairness.